

LECTURE - 3

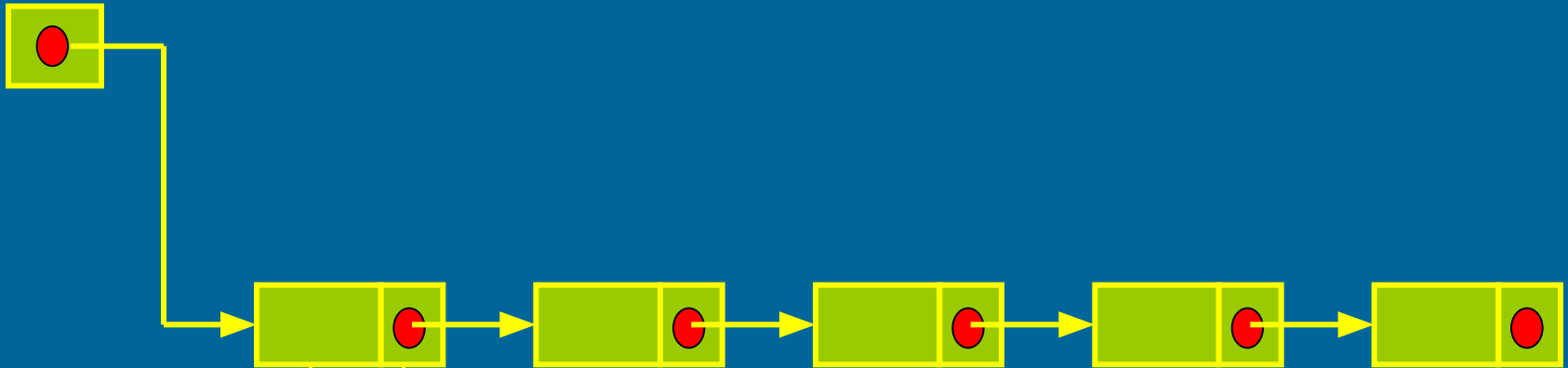
What is an Linked List?

A linked list, or one-way list, is a linear collection of data elements, called nodes where the linear order is given by means of pointers

Each node is divided into two parts:

1. The first part contains the information of the element
2. The second part, called the **link field** or **nextpointer field** contains the address of the next node

start



Nextpointer field of 1st node

Information part of 1st node

INFO

LINK

START

9

1

2

3

4

5

6

7

8

9

10

11

12

O

T

X

N

I

E

6

0

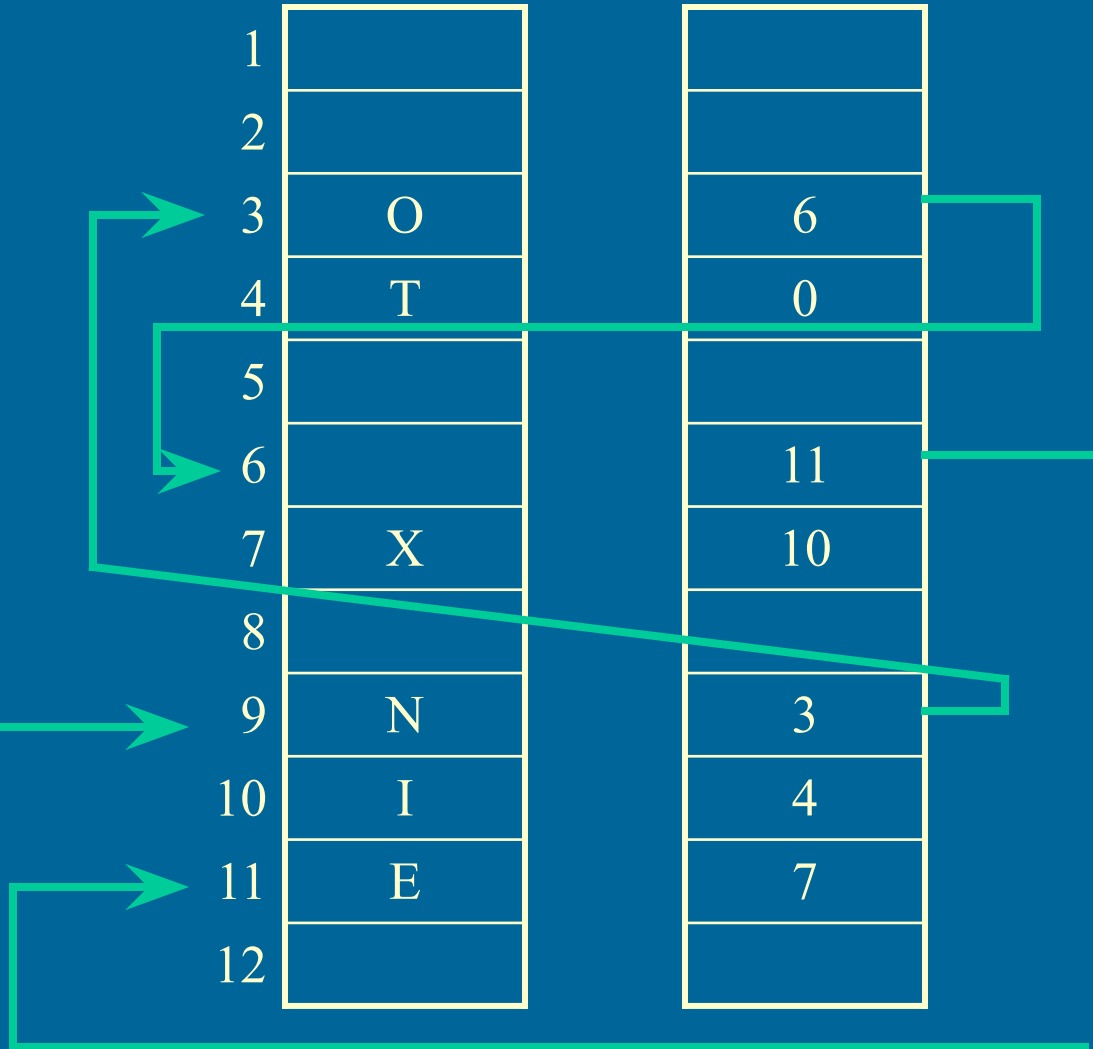
11

10

3

4

7



PRACTICE

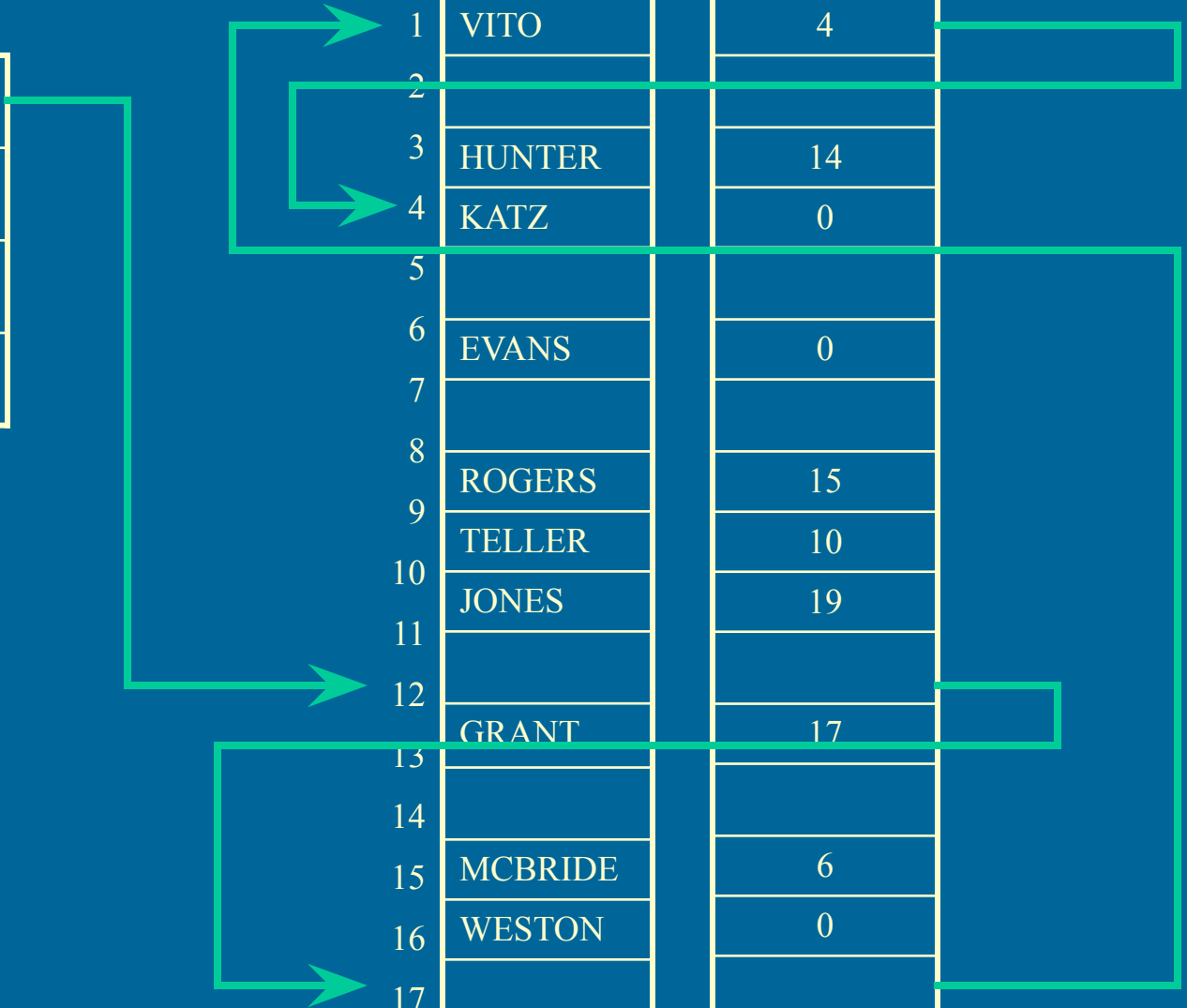
Example 5.2

ON PAGE 117

BROKER	
BOND	
KELLY	
HALL	
NELSON	

POINT	
12	
3	
0	
9	

CUSTOMER		LINK	
1	VITO	4	
2			
3	HUNTER	14	
4	KATZ	0	
5			
6	EVANS	0	
7			
8			
9	ROGERS	15	
10	TELLER	10	
11	JONES	19	
12			
13	GRANT	17	
14			
15	MCBRIDE	6	
16	WESTON	0	
17			
18	SCOTT	1	
19			



Bond's list of customers

GRANT, SCOTT, VITO, KATZ

Kelly's list of customers

HUNTER, MCBRIDE, EVANS

Nelson's list of customers

TELLER, JONES, ADAMS,
ROGERS, WESTON

PRACTICE

Example 5.3

ON PAGE 117

PRACTICE

Example 5.5

ON PAGE 119

Algorithm-1

Traversing a Linked List

1. Set PTR := Start [initializes pointer PTR]
2. Repeat steps 3 and 4 while PTR $\neq$ NULL
3. Apply process to INFO[PTR]
4. Set PTR := LINK[PTR]
- [End of loop]
5. Exit

Searching a Linked List

There are two searching algorithms for finding the location LOC of the node where item first appears in the LIST.

1. The first algorithm does not assume that the data in LIST are sorted.
2. The second algorithm does assume that LIST is sorted.

Algorithm-2

SEARCH (Info, Link, Start, Item, Loc)

Searching in a Linked List

1. Set PTR := START
2. Repeat steps 3 while PTR $\neq$ NULL
3. If ITEM = INFO[PTR], then:
 Set LOC := PTR, and Exit
 Else:
 Set PTR := LINK[PTR]
 [End of if structure]
[End of loop]
4. Set LOC := NULL [search is unsuccessful]
5. Exit

Algorithm-3

SEARCHSL (Info, Link, Start, Item, Loc)

Searching in a sorted Linked List

1. Set PTR := Start
2. Repeat steps 3 while PTR $\neq$ NULL
3. If ITEM > INFO[PTR], then:
 Set PTR := LINK[PTR]
 Else if ITEM = INFO[PTR], then:
 Set LOC := PTR, and Exit
 Else:
 Set LOC := NULL, and Exit
 [End of if structure]
- [End of loop]
4. Set LOC := NULL [search is unsuccessful]
5. Exit

MEMORY ALLOCATION

The maintenance of linked lists in memory assumes the possibility of inserting new nodes into the linked list and some mechanism is required whereby the memory space of deleted nodes becomes available for future use.

A special list is maintained which consists of unused memory cells. This list, which has its own pointer is called the list of available space or the free-storage list or the free pool.

MEMORY ALLOCATION....(cont)

Linked list are implemented by parallel arrays.

Suppose insertions and deletions are to be performed on our linked list. Then the unused memory cells in the arrays will also be linked together to form a linked list using AVAIL as its pointer variable

This free-storage list is also called AVAIL list.

START

5

INFO

LINK

1

KIRK

7

2

6

3

DEAN

11

4

MAXWELL

12

5

ADAMS

3

6

0

7

LANE

4

8

GREEN

1

9

SAMUELS

0

10

2

11

FIELDS

8

12

NELSON

9

AVAIL

10

PRACTICE

Example 5.11

(a, b, c)

ON PAGE 124

GARBAGE COLLECTION

The operating system of a computer may periodically collect all the deleted space onto the free-storage list. Any technique which does this collection is called garbage collection.

Garbage collection usually takes place in two steps.

1. First the computer runs through all list, tagging those cells which are currently in use.
2. Second the computer runs through memory, collecting all untagged space onto the free storage list.

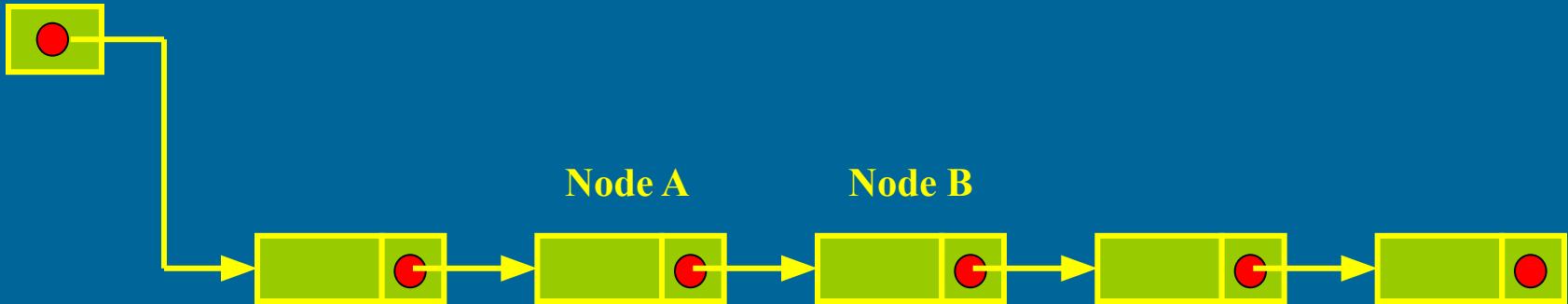
OVERFLOW & UNDERFLOW

Sometimes new data are to be inserted into a data structures but there is no available space. This situation is usually called **overflow**.

The term **underflow** refers to the situation where one wants to delete data from a data structure that is empty.

Insertion into a Linked List

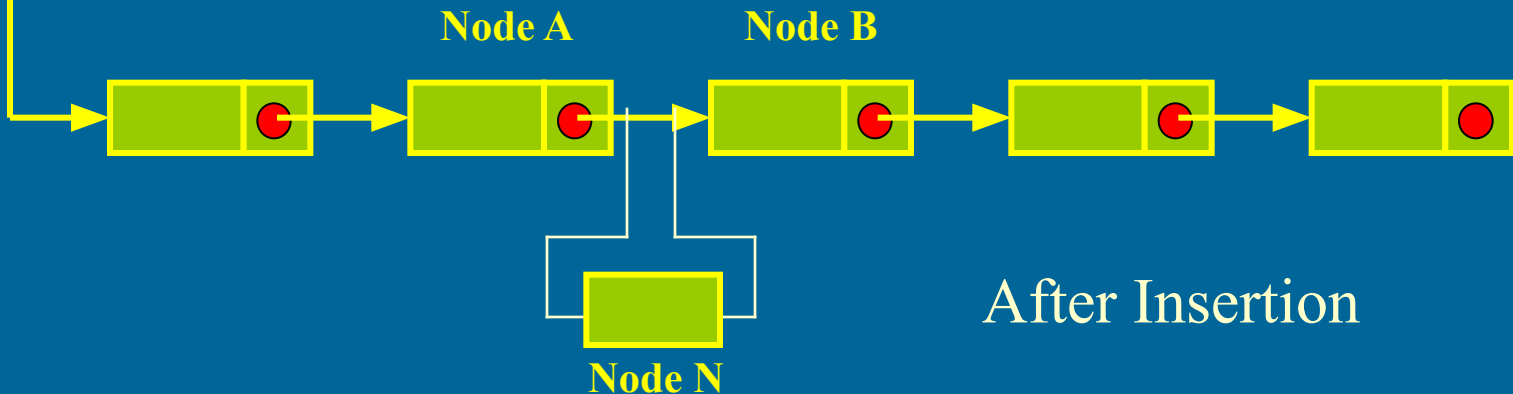
start



Before Insertion



start



After Insertion

Figure in last slide does not take into account that the memory space for the new node N will come from the AVAIL list.

Specifically, for easier processing, the first node in the AVAIL list will be used for the new node N.

Observe that three pointer fields are changed as follows:

1. The nextpointer field of node A now points to the new node N, to which AVAIL previously pointed
2. Avail now points to the second node in the free pool, to which node N previously pointed
3. The nextpointer field of node N now points to node B, to which node A previously pointed

Algorithm-4

INSFIRST (Info, Link, Start, Avail, Item)

Inserting at the beginning of a Linked List

1. [OVERFLOW] if AVAIL := NULL then write : overflow and Exit
2. [Remove first node from AVAIL list]
Set New := AVAIL and AVAIL := LINK[AVAIL]
3. Set INFO[NEW] := ITEM [copies new data into new node]
4. Set LINK[NEW] := START [new node now points to original first node]
5. Set START := NEW [changes start so it points to the new node]
6. Exit

Algorithm-5

INSLOC (Info, Link, Start, Avail, Loc, Item)

Inserting after a given node

1. [OVERFLOW] if AVAIL := NULL then write : overflow and Exit
2. [Remove first node from AVAIL list]
Set New := AVAIL and AVAIL := LINK[AVAIL]
3. Set INFO[NEW] := ITEM [copies new data into new node]
4. If LOC = NULL, then;
set LINK[NEW] := START and START := NEW
Else:
set LINK[NEW] := LINK[LOC] and LINK[LOC] := NEW
[end of IF structure]
5. Exit